

MATERIALIZACIÓN DE CASOS DE USO CON SPRING BOOT

OBJETIVO

En la **Guía 06** implementamos los casos de uso como **POJOs puros**: clases Java sin dependencia de ningún framework, con repositorios en memoria basados en HashMap . Esto nos permitió concentrarnos en la orquestación y la separación de responsabilidades.

En las **Guías 07 y 08** diseñamos la API REST e implementamos Controllers que delegan al dominio mediante DTOs y Mappers.

Ahora es momento de completar la integración:

Transformar los casos de uso de la Guía 06 en componentes gestionados por Spring Boot, conectándolos con el framework para que funcionen en un entorno real de aplicación.

¿Por qué es necesario este paso?

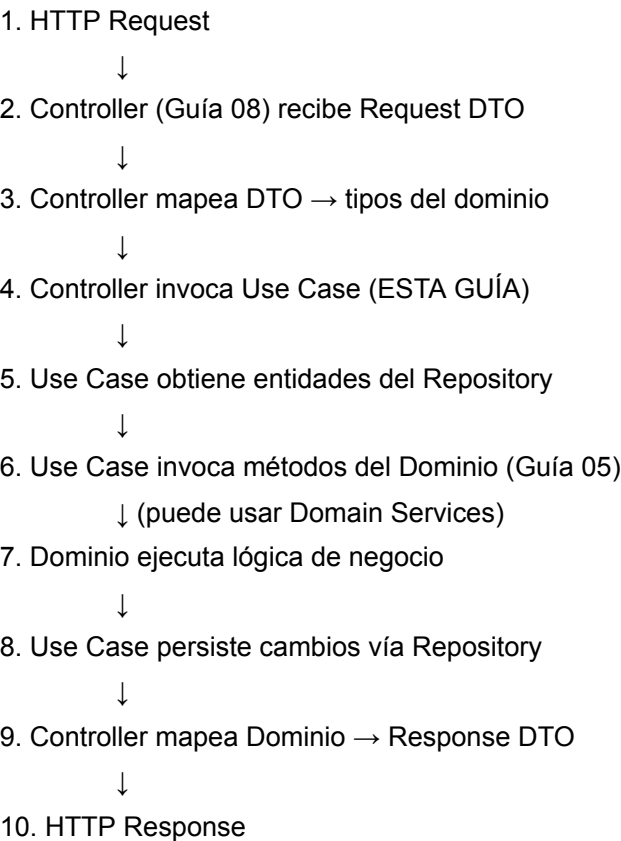
1. **Inyección de dependencias:** Spring gestiona la creación e interconexión de los objetos, eliminando la instanciación manual.
2. **Gestión de transacciones:** @Transactional asegura consistencia cuando una operación involucra múltiples escrituras.
3. **Integración con infraestructura:** Los repositorios en memoria serán reemplazados por implementaciones reales (Guía 10), y Spring necesita conocer nuestros servicios.
4. **Testing**

profesional: Mockito y el contexto de Spring permiten escribir tests unitarios robustos.

💡💡 **Principio clave:** La capa de aplicación orquesta, el dominio decide. Spring solo facilita la conexión entre las piezas.

RECORDATORIO: CAPAS Y RESPONSABILIDADES

Este es el flujo completo que hemos construido a lo largo de las guías:



Capa	Responsabilidad	Conoce DTOs	Guía
Controller	Adaptar HTTP ↔ Dominio	✓ Sí	08
Use Case	Orquestrar el dominio	✗ No	06, 09
Dominio	Reglas e invariantes	✗ No	04, 05

💡💡 **Importante:** Los Use Cases reciben **tipos del dominio o value objects**, no DTOs. El mapeo DTO → dominio ocurre en el Controller (Guía 08). De esta manera no acoplamos la capa de aplicación al formato de la API.

ADAPTACIÓN DE INTERFACES DE REPOSITORIO

En la Guía 06, definimos las interfaces de repositorio usando **lenguaje del dominio**:

```
// Guía 06: lenguaje ubicuo
public interface SolicitudRepository {
    Solicitud obtenerPorCodigo(CodigoSolicitud codigo);
    void guardar(Solicitud solicitud);
    List<Solicitud> obtenerPorEstado(EstadoSolicitud estado);
}
```

Al integrar con **Spring Data**, conviene adaptar estas interfaces a las **convenciones de Spring**, que permiten auto-generar implementaciones a partir del nombre del método:

```
// Guía 09: adaptado a convenciones de Spring Data
public interface SolicitudRepository {
    Solicitud save(Solicitud solicitud);
    Optional<Solicitud> findById(String id);
    Optional<Solicitud> findByCodigo(CodigoSolicitud codigo);
    List<Solicitud> findByEstado(EstadoSolicitud estado);
}
```

¿Por qué este cambio?

1. **Auto-generación:** Spring Data puede generar la implementación automáticamente si los nombres siguen la convención `findBy...` , `save` , `deleteBy...` .
2. **Preparación para la Guía 10:** Cuando reemplacemos los repositorios en memoria por Spring Data JPA, estas firmas serán directamente compatibles.
3. **Pragmatismo:** Sacrificamos algo de lenguaje ubicuo por compatibilidad con el framework, una decisión técnica común en proyectos reales.

◆◆ **Decisión consciente:** Este cambio es una concesión al framework. En un proyecto donde el lenguaje ubicuo sea crítico, los métodos del dominio podrían mantener nombres en español y el adaptador de infraestructura haría la traducción.

PARTE 1: DE POJO A SERVICIO SPRING

Antes: Guía 06 (POJO puro)

// Guía 06: sin framework, repositorio inyectado manualmente

```
public class CrearSolicitudUseCase {
    private final SolicitudRepository repository;
    private final GeneradorCodigo generadorCodigo;

    public CrearSolicitudUseCase(SolicitudRepository repository,
                                GeneradorCodigo generadorCodigo) {

        this.repository = repository;
        this.generadorCodigo = generadorCodigo;
    }

    public Solicitud ejecutar(TipoSolicitud tipo, String descripcion,
                              Usuario solicitante) {
        CodigoSolicitud codigo = generadorCodigo.generar();
        Solicitud solicitud = Solicitud.crear(codigo, tipo, descripcion, solicitante); repository.guardar(solicitud);
        return solicitud;
    }
}
```

Después: Guía 09 (Servicio Spring)

Agregamos tres anotaciones:

@Service → Spring gestiona la instancia

@RequiredArgsConstructor → Lombok genera el constructor (eliminamos boilerplate)

@Transactional → Spring gestiona la transacción

```
package co.edu.uniquindio.solicitudes.application.usecase;
```

```
import co.edu.uniquindio.solicitudes.domain.entity.Solicitud; import
```

```
co.edu.uniquindio.solicitudes.domain.valueobject.*;
```

```
import co.edu.uniquindio.solicitudes.domain.entity.Usuario; import
```

```
co.edu.uniquindio.solicitudes.domain.repository.SolicitudRepository; import
```

```
co.edu.uniquindio.solicitudes.domain.service.GeneradorCodigosService; import
```

```
lombok.RequiredArgsConstructor;
```

```
import org.springframework.stereotype.Service;
```

```
import org.springframework.transaction.annotation.Transactional;
```

```
@Service
```

```
@RequiredArgsConstructor
```

```
public class CrearSolicitudUseCase {
```

```
    private final SolicitudRepository solicitudRepository;
```

```

private final GeneradorCodigosService generadorCodigos;

@Transactional
public Solicitud ejecutar(TipoSolicitud tipo, String descripcion, Usuario solicitante,
                        CanalOrigen canal) {
    // 1. Generar código único (Domain Service)
    CodigoSolicitud codigo = generadorCodigos.generarNuevo();

    // 2. Crear solicitud (DOMINIO ejecuta la lógica)
    Solicitud solicitud = Solicitud.crear(codigo, tipo, descripcion, solicitante, canal);

    // 3. Persistir
    return solicitudRepository.save(solicitud);
}
}

```

Observa:

El Use Case **no recibe DTOs**, recibe tipos del dominio.

El Controller (Guía 08) hizo la conversión DTO → dominio **antes** de llamar aquí. La lógica del Use Case es idéntica a la de la Guía 06, solo cambia la infraestructura.

¿Cómo se conecta con el Controller?

El Controller de la Guía 08 sigue siendo el responsable de mapear DTOs:

// Controller (Guía 08) — el DTO se procesa aquí, no en el Use Case @PostMapping

```

public ResponseEntity<SolicitudDetalleResponse> crearSolicitud(
    @Valid @RequestBody CrearSolicitudRequest request) {

    // 1. Obtener tipos del dominio
    TipoSolicitud tipo = tipoSolicitudRepository
        .findById(request.tipoSolicitudId())
        .orElseThrow(() -> new TipoSolicitudNoEncontradoException(
            request.tipoSolicitudId()));

    // 2. Delegar al Use Case con tipos del dominio
    Solicitud solicitud = crearSolicitudUseCase.ejecutar(
        tipo, request.descripcion(), usuarioActual, request.canalOrigen() );

    // 3. Mapear dominio → DTO respuesta
    SolicitudDetalleResponse response = mapper.toDetalleResponse(solicitud);
}

```

```

    URI location = ServletUriComponentsBuilder
        .fromCurrentRequest().path("/{id}")
        .buildAndExpand(solicitud.getId()).toUri();

    return ResponseEntity.created(location).body(response);
}

```

PARTE 2: @TRANSACTIONAL

¿Qué es @Transactional?

@Transactional define un **límite transaccional**: si todas las operaciones dentro del método se completan, se hace **commit**. Si alguna lanza una excepción, se hace **rollback** automáticamente.

```
@Service
```

```
@RequiredArgsConstructor
```

```

public class AsignarResponsableUseCase {

    private final SolicitudRepository solicitudRepository;
    private final UsuarioRepository usuarioRepository;

```

```
/**
```

```

 * @Transactional asegura que:
 * 1. Se inicia una transacción al entrar
 * 2. Se hace commit si completa exitosamente
 * 3. Se hace rollback si lanza excepción
 */

```

```
@Transactional
```

```

    public Solicitud ejecutar(String solicitudId, String responsableId, String coordinadorId) {
        Solicitud solicitud = solicitudRepository.findById(solicitudId) .orElseThrow(() -> new
            SolicitudNoEncontradaException(solicitudId));

        Usuario responsable = usuarioRepository.findById(responsableId) .orElseThrow(() -> new
            UsuarioNoEncontradoException(responsableId));

        Usuario coordinador = usuarioRepository.findById(coordinadorId) .orElseThrow(() -> new
            UsuarioNoEncontradoException(coordinadorId));
    }

```

```

        // Si asignar() lanza excepción del dominio, se hace rollback
        solicitud.asignarResponsable(responsable, coordinador);

        return solicitudRepository.save(solicitud);
    }
}

```

readOnly para operaciones de solo lectura

Cuando un Use Case solo consulta datos sin modificarlos, se puede optimizar con

readOnly = true :

```

@Service
@RequiredArgsConstructor
public class ObtenerSolicitudUseCase {

    private final SolicitudRepository solicitudRepository;

    @Transactional(readOnly = true)
    public Solicitud ejecutar(String solicitudId) {
        return solicitudRepository.findById(solicitudId)
            .orElseThrow(() -> new SolicitudNoEncontradaException(solicitudId));
    }
}

```

Reglas de uso

Regla	Razón
@Transactional solo en Use Cases	Son el límite natural de una operación de negocio
Nunca en Controllers	El Controller no conoce transacciones
Nunca en el Dominio	El Dominio no depende de infraestructura
readOnly = true en queries	Optimiza rendimiento, evita locks

 **Nota sobre múltiples agregados:** En la Guía 05 establecimos que un agregado = una transacción. Si una operación necesita modificar dos agregados distintos (ej. cerrar solicitud Y crear notificación), la solución correcta es usar **eventos de dominio**. Para esta guía, mantendremos la simplicidad de un solo agregado por transacción. Los eventos de dominio son un patrón avanzado que queda fuera del alcance del curso.

PARTE 3: USE CASES CON DOMAIN SERVICES

Cuando la lógica de negocio **no pertenece a una sola entidad** (Guía 05), usamos un **Domain Service**. El Use Case orquesta la interacción entre el agregado y el servicio de dominio.

Ejemplo: Clasificar Solicitud con Validación Cruzada

Supongamos que clasificar una solicitud requiere validar que el tipo asignado sea coherente con el canal de origen (una regla que involucra varios conceptos):

```
package co.edu.uniquindio.solicitudes.domain.service;

/**
 * DOMAIN SERVICE: Valida coherencia entre tipo y canal
 * Vive en domain/service/ — no depende de infraestructura
 */
public class ValidadorClasificacionService {

    /**
     * Verifica que la combinación tipo-canal sea válida
     * según las reglas del negocio
     */
    public void validarCoherencia(TipoSolicitud tipo, CanalOrigen canal) { // Ejemplo: solicitudes de
        homologación solo por correo o CSU
        if (tipo.esHomologacion() && !canal.esCanalesFormal()) {
            throw new ReglaDominioException(
                "Las solicitudes de homologación solo se reciben por canales formales") }
        }
    }
}
```

Use Case que usa el Domain Service

```
@Service
@RequiredArgsConstructor
public class ClasificarSolicitudUseCase {

    private final SolicitudRepository solicitudRepository;
    private final ValidadorClasificacionService validadorClasificacion;
```

@Transactional

```
public Solicitud ejecutar(String solicitudId, TipoSolicitud nuevoTipo, Prioridad prioridad, Usuario coordinador) {  
    // 1. Obtener solicitud  
    Solicitud solicitud = solicitudRepository.findById(solicitudId)  
        .orElseThrow(() -> new SolicitudNoEncontradaException(solicitudId));  
  
    // 2. DOMAIN SERVICE valida coherencia  
    validadorClasificacion.validarCoherencia(nuevoTipo, solicitud.getCanalOrigen())  
  
    // 3. DOMINIO ejecuta la clasificación (valida estado, registra historial) solicitud.clasificar(nuevoTipo, prioridad, coordinador);  
  
    // 4. Persistir  
    return solicitudRepository.save(solicitud);  
}
```

Observa la distribución:

Domain Service → lógica que cruza conceptos (ValidadorClasificacionService) **Entidad** → reglas de su propio estado (solicitud.clasificar() valida que esté en estado REGISTRADA)

Use Case → orquesta ambos, sin decidir nada

◆◆ **Nota:** La validación de **permisos** (¿puede este usuario clasificar?) no es responsabilidad del Use Case ni del dominio. Se implementará con Spring Security en la Guía 13.

PARTE 4: SEPARACIÓN DE OPERACIONES DE LECTURA Y ESCRITURA

Es buena práctica separar operaciones que **modifican estado** (Commands) de las que **solo consultan** (Queries).

Commands (Modifican estado)

@Service

@RequiredArgsConstructor

```
public class CerrarSolicitudUseCase {
```

```
    private final SolicitudRepository solicitudRepository;
```

```
    @Transactional
```

```
    public Solicitud ejecutar(String solicitudId, String observacion, Usuario responsable) {
```

```
        Solicitud solicitud = solicitudRepository.findById(solicitudId) .orElseThrow(() -> new
```

```
            SolicitudNoEncontradaException(solicitudId));
```

```
        solicitud.cerrar(observacion, responsable);
```

```
        return solicitudRepository.save(solicitud);
```

```
    }
```

```
}
```

Queries (Solo lectura)

@Service

@RequiredArgsConstructor

```
public class ListarSolicitudesUseCase {
```

```
    private final SolicitudRepository solicitudRepository;
```

```
    @Transactional(readOnly = true)
```

```
    public List<Solicitud> ejecutar(EstadoSolicitud estado) {
```

```
        if (estado != null) {
```

```
            return solicitudRepository.findByEstado(estado);
```

```
        }
```

```
        return solicitudRepository.findAll();
```

```
    }
```

```
}
```

¿Por qué separarlos?

Aspecto	Commands	Queries
Transacción	@Transactional	@Transactional(readOnly = true)
Modifica estado	Sí	No
Complejidad	Mayor (orquestación + persistencia)	Menor (solo consulta)
Ejemplo	CrearSolicitudUseCase	ListarSolicitudesUseCase

◆◆ **Referencia:** Este principio se conoce formalmente como **CQS** (Command-Query Separation). Para profundizar, consulte: Fowler, M. - *CommandQuerySeparation*.

PARTE 5: TESTING DE USE CASES

Test Unitario con Mockito

Los tests de Use Cases usan **mocks** para los repositorios, permitiendo probar la orquestación sin base de datos:

```
package co.edu.uniquindio.solicitudes.application.usecase;
```

```
import co.edu.uniquindio.solicitudes.domain.entity.*;
import co.edu.uniquindio.solicitudes.domain.repository.*;
import co.edu.uniquindio.solicitudes.domain.valueobject.*;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
```

```
import java.util.Optional;
```

```
import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.*;
```

```
@ExtendWith(MockitoExtension.class)
```

```
class CerrarSolicitudUseCaseTest {
```

```
    @Mock
```

```
    private SolicitudRepository solicitudRepository;
```

```
    @InjectMocks
```

```
    private CerrarSolicitudUseCase useCase;
```

```
    @Test
```

```

void deberiaCerrarSolicitudCuandoEstadoValido() {
    // Arrange
    String solicitudId = "123";
    Solicitud solicitud = crearSolicitudAtendida();
    Usuario responsable = crearResponsableMock();

    when(solicitudRepository.findById(solicitudId))
        .thenReturn(Optional.of(solicitud));
    when(solicitudRepository.save(any()))
        .thenReturn(solicitud);

    // Act

    Solicitud resultado = useCase.ejecutar(solicitudId, "Resuelta", responsable);

    // Assert
    assertNotNull(resultado);
    verify(solicitudRepository).findById(solicitudId);
    verify(solicitudRepository).save(solicitud);
}

```

@Test

```

void deberiaLanzarExcepcionCuandoSolicitudNoExiste() {
    // Arrange
    when(solicitudRepository.findById(any()))
        .thenReturn(Optional.empty());

    // Act & Assert
    assertThrows(
        SolicitudNoEncontradaException.class,
        () -> useCase.ejecutar("999", "Observación", crearResponsableMock()) );

    // Verify: nunca se intentó guardar
    verify(solicitudRepository, never()).save(any());
}

```

// Métodos auxiliares para crear objetos de prueba

```

private Solicitud crearSolicitudAtendida() {
    // Crear una solicitud en estado ATENDIDA con responsable asignado
    // usando los métodos de la entidad (Guía 05)
    // ...
}

```

```

private Usuario crearResponsableMock() {

```

```
}  
    // ...  
}  
}
```

¿Qué se testea en un Use Case?

Aspecto	Se testea	Ejemplo
Orquestación correcta	✓	Se llama findById antes de save
Manejo de errores	✓	Excepción cuando no existe
Delegación al dominio	✓	Se invoca solicitud.cerrar()
Reglas de negocio	✗	Eso se testea en el dominio (Guía 05.1)

EJERCICIOS PRÁCTICOS

Ejercicio 1: Transformar Use Cases de la Guía 06

Transforme los 5 casos de uso implementados en la Guía 06 en servicios Spring:

1. CrearSolicitudUseCase
2. AsignarResponsableUseCase
3. CambiarEstadoUseCase
4. CerrarSolicitudUseCase
5. ConsultarSolicitudesPorEstadoUseCase

Requisitos:

Agregar @Service y @RequiredArgsConstructor

Agregar @Transactional (o @Transactional(readOnly = true) para queries) Los Use

Cases deben recibir **tipos del dominio**, no DTOs

Adaptar las interfaces de repositorio a convenciones Spring Data (save , findById ,
findByEstado)

Ejercicio 2: Conectar con Controllers

Actualice los Controllers de la Guía 08 para que:

Hagan la conversión DTO → tipos del dominio
Deleguen a los Use Cases con tipos del dominio
Mapeen la respuesta dominio → DTO de salida

Ejercicio 3: Escribir Tests

Para al menos 2 Use Cases, escriba tests unitarios con Mockito que verifiquen:

El flujo exitoso

El caso de error (entidad no encontrada)

ESTRUCTURA DEL PROYECTO

Así queda la estructura después de esta guía:

```
src/main/java/co/edu/uniquindio/solicitudes/  
├── domain/  
│   ├── entity/  
│   │   ├── Solicitud.java ← Guía 05  
│   │   └── Usuario.java  
│   ├── valueobject/  
│   │   ├── CodigoSolicitud.java ← Guía 04  
│   │   ├── EstadoSolicitud.java  
│   │   └── Prioridad.java  
│   └── repository/  
│       ├── SolicitudRepository.java ← Guía 06, adaptado aquí | | └──  
UsuarioRepository.java  
│   ├── service/  
│   │   └── GeneradorCodigosService.java  
│   └── exception/  
│       └── ReglaDominioException.java  
│  
├── application/  
│   └── usecase/  
│       ├── CrearSolicitudUseCase.java ← ESTA GUÍA | | └──  
AsignarResponsableUseCase.java  
│   ├── CerrarSolicitudUseCase.java  
│   ├── ClasificarSolicitudUseCase.java  
│   ├── ObtenerSolicitudUseCase.java  
│   └── ListarSolicitudesUseCase.java  
│  
└── infrastructure/
```

```
└─ rest/
    └─ SolicitudController.java ← Guía 08
    └─ mapper/
        └─ SolicitudMapper.java
    └─ exception/
        └─ GlobalExceptionHandler.java
```

PRECAUCIONES Y RECOMENDACIONES

1. **Use Cases no reciben DTOs:** Los DTOs se procesan en el Controller. El Use Case trabaja con tipos del dominio.
2. **Un Use Case = una transacción:** No dividir transacciones entre múltiples Use Cases.
3. **Use Cases pequeños:** Cada uno hace UNA cosa.
4. **No lógica de negocio:** Toda regla va al dominio (Guía 05) o a Domain Services. 5.
6. **@Transactional solo en Use Cases:** No en Controllers ni en entidades de dominio.
6. **Inyección por constructor:** Facilita testing con Mockito.

EVALUACIÓN O RESULTADO

Al finalizar, los estudiantes deben:

1. ☒ Haber transformado todos los Use Cases de la Guía 06 en servicios Spring
2. ☒ Comprender la diferencia práctica entre Application Service y Domain Service
3. ☒ Gestionar transacciones con @Transactional
4. ☒ Conectar Use Cases con Controllers (sin filtrar DTOs al Use Case)
5. ☒ Escribir tests unitarios de Use Cases con Mockito

PRÓXIMA ACTIVIDAD

En la **Guía 10: Persistencia con H2 y Patrón Repository** implementaremos las interfaces de repositorio con Spring Data JPA:

1. **Preparar:** Verificar que las interfaces de repositorio estén adaptadas a convenciones Spring Data.
2. **Investigar:** Conceptos básicos de bases de datos relacionales y JPA.

3. **Leer:** Fowler, M. - "Repository Pattern"

Los Use Cases que construiste hoy no cambiarán cuando agreguemos persistencia. Solo cambiarán las implementaciones de los repositorios.

REFERENCIAS BIBLIOGRÁFICAS

1. Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison Wesley.
2. Vernon, V. (2013). *Implementing Domain-Driven Design*. Addison-Wesley.
3. Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
4. Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.
5. Spring Framework Documentation: [Dependency Injection](#)
6. Lombok Project: <https://projectlombok.org/>